

MATH3881/5881: Statistical Machine Learning Theory

Week 5: Convolutional Neural Networks

Sarat Moka

UNSW, Sydney

Key Topics

5.1	Motivation: From FNNs to CNNs	5-2
5.2	The Convolution Operation	5-6
5.3	From Fully Connected Layers to Convolutional Layers	5-8
5.4	Building a Convolutional Layer	5-9
5.5	CNN Architectures: Pooling, VGG19, and ResNets	5-14

Books:

- (A) *Mathematical Engineering of Deep Learning* book by Liquet, Moka, and Nazarathy (2024), Chapters 5 and 6.

5.1 Motivation: From FNNs to CNNs

In Weeks 2 and 3 we studied *feedforward neural networks* (FNNs) and the optimisation methods used to train them. FNNs are general-purpose function approximators: each neuron in a layer is connected to *every* neuron in the next layer, yielding weight matrices $W^{[\ell]}$ of dimension $N^{[\ell]} \times N^{[\ell-1]}$. For an input image of dimension $3 \times 224 \times 224 \approx 150,000$ features, even a single-layer dense classifier for $K = 1000$ classes requires ≈ 150 million parameters. Deeper dense networks require even more. This highlights two practical limitations of FNNs for image data:

- **Too many parameters.** Most parameters serve no useful inductive purpose; the model is susceptible to overfitting and is computationally intractable.
- **Structure-agnostic.** Flattening an image into a vector discards all spatial relationships between pixels.

Exercise 5.1.1 (Parameter Count of a Dense Image Classifier)

Verify the ≈ 150 -million figure quoted above. A single-layer dense classifier maps a $3 \times 224 \times 224$ colour image, flattened to a vector of length $3 \cdot 224 \cdot 224 = 150,528$, to a $K = 1000$ -class probability vector via a weight matrix $W \in \mathbb{R}^{1000 \times 150,528}$ and a bias vector $b \in \mathbb{R}^{1000}$.

- Compute the total number of learnable parameters in (W, b) .
- At 4 bytes per parameter (32-bit float), how much memory (in MB) does this single layer require?

Two Key Structural Properties of Image Data

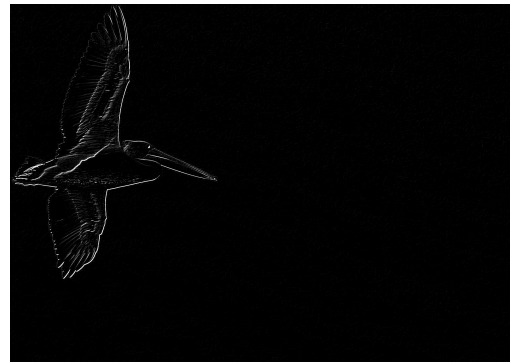
Images exhibit two properties that any good model should exploit:

Translation invariance. The label of an image is independent of where the object appears. A cat is a cat whether it appears at the top-left or bottom-right of the frame. Consequently, the *rule* for detecting a cat should be the same at every spatial location. Figure 5.1 illustrates this with a fixed edge filter applied to two images of the same pelican at different positions: the output simply moves with the input.

Locality. The presence of a feature at one location depends mostly on nearby pixels, not on pixels far away. Low-level features (edges, corners) are determined by patches of 3×3 to 7×7 pixels; only higher-level features need broader context. Figure 5.2 makes this concrete: everything we need to identify a bird is contained inside the red box around it.



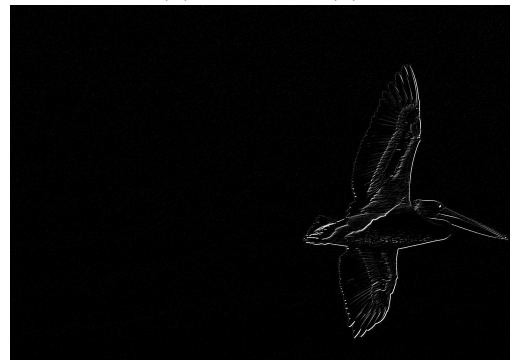
(a) Input (a): top-left



(b) Output of (a)



(c) Input (c): bottom-right

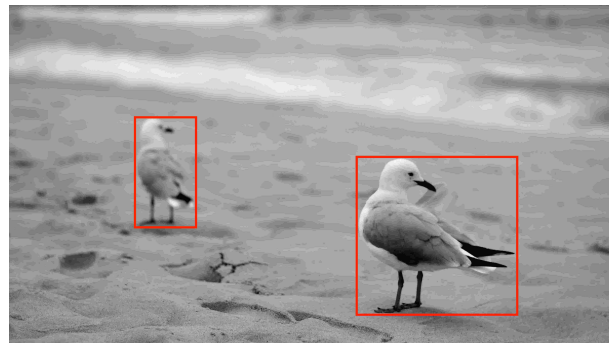


(d) Output of (c)

Figure 5.1: Translation invariance illustrated with an edge filter applied to a pelican. The pair (a, b) and the pair (c, d) differ only by the position of the pelican; the filter's output simply tracks the position of the input. The filter's *action* on the object is independent of *where* the object lies in the frame.



(a) Pelicans



(b) Seagulls

Figure 5.2: Locality. The pixel information inside each red box is by itself enough to identify the bird inside. Far-away pixels are essentially irrelevant. At a finer scale, the same is true for low-level features (edges, corners), which depend on only a few neighbouring pixels.

Convolutional neural networks (CNNs) are designed to enforce both properties through *parameter sharing* (translation invariance) and *sparse connectivity* (locality). As we derive in Section 5.3, these two constraints together reduce the number of parameters from billions to thousands per layer.

Edge Detection as a Motivating Example

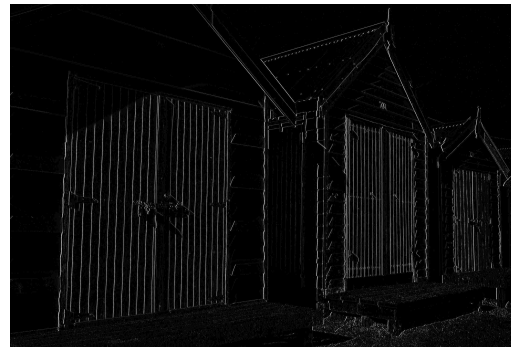
Before defining CNNs formally, it is instructive to recall how *classical* image filters exploit the same two properties. The **Sobel filter** detects edges by convolving an image x with two small 3×3 kernels:

$$W^{(\leftrightarrow)} = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}, \quad W^{(\updownarrow)} = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad (5.1)$$

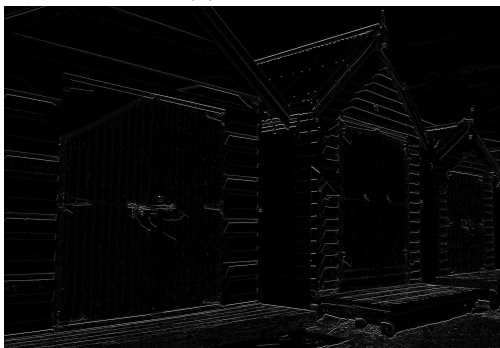
for horizontal and vertical edge detection respectively. These 3×3 kernels are *shared across all spatial locations* (translation invariance) and look at only a 3×3 neighbourhood (locality). **In a CNN, the entries of such kernels are not hand-engineered but are learned from data during training.**



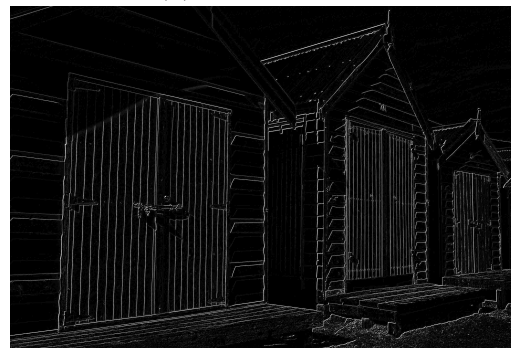
(a) Original



(b) Vertical edges



(c) Horizontal edges



(d) Combined

Figure 5.3: Edge detection via the Sobel filter (5.1). Each output is a fixed linear transformation (convolution) of the input applied identically at every spatial location.

Example 5.1.1

Take a tiny 3×3 patch of an image where the top is dark and the bottom is bright (a horizontal edge running across the middle):

$$x = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}.$$

“Applying the horizontal Sobel filter $W^{(\leftrightarrow)}$ at this location” means: form the element-wise product of $W^{(\leftrightarrow)}$ and x , then sum the nine entries:

$$\sum_{i,j} W_{i,j}^{(\leftrightarrow)} x_{i,j} = (-1)(0) + (-2)(0) + (-1)(0) + 0 + 0 + 0 + (1)(1) + (2)(1) + (1)(1) = 4.$$

A large positive number flags an edge. On a flat patch (all entries equal to, say, 0.5), the top-row negatives cancel the bottom-row positives, giving 0: no edge response.

Figure 5.3(c) is obtained by repeating exactly this 3×3 calculation with $W^{(\leftrightarrow)}$ sitting over every 3×3 window of the input image. We formalise this sliding-window operation as *2D convolution* in Section 5.2.

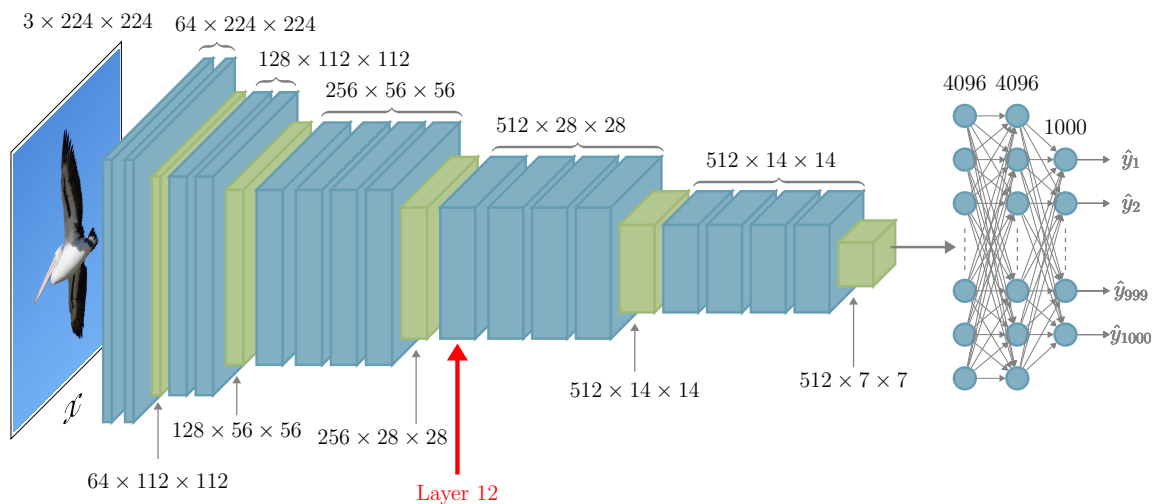


Figure 5.4: The VGG19 architecture. The input is a $3 \times 224 \times 224$ colour image processed through convolutional and pooling layers, then dense layers, to produce a 1000-class probability vector.

The VGG19 Network: A Preview

To get a feel for what a complete CNN looks like, consider the VGG19 architecture (Figure 5.4), which accepts a $3 \times 224 \times 224$ colour image and outputs a probability vector over $K = 1000$ classes.

It has ≈ 144 million parameters (similar to a single-layer dense classifier) yet spans 19 *trainable* layers of convolutional and dense operations. Its depth is what makes it expressive; each layer learns progressively higher-level features: early layers detect edges and textures, later layers detect object parts and whole objects. We return to VGG19 in full detail in Section 5.5.

5.2 The Convolution Operation

The convolution operation is the mathematical engine of CNNs. We develop it from first principles in one and two dimensions.

One-Dimensional Convolution

Let $f, g : \mathbb{Z} \rightarrow \mathbb{R}$ be two discrete sequences. Their **(discrete) convolution** is

$$(f \star g)(t) = \sum_{\tau \in \mathbb{Z}} f(t - \tau) g(\tau), \quad t \in \mathbb{Z}. \quad (5.2)$$

At each t , one sequence is *flipped* (reflected about the origin) and *shifted* by t , then multiplied pointwise with the other and summed. The operation is **commutative**: $(f \star g)(t) = (g \star f)(t)$.

Two-Dimensional Convolution

For image data, let $W = [w_{i,j}] \in \mathbb{R}^{K_h \times K_v}$ be a **kernel** and $x = [x_{i,j}] \in \mathbb{R}^{M_h \times M_v}$ be an image with $K_h \leq M_h$, $K_v \leq M_v$. The **2D convolution** used in deep learning is¹

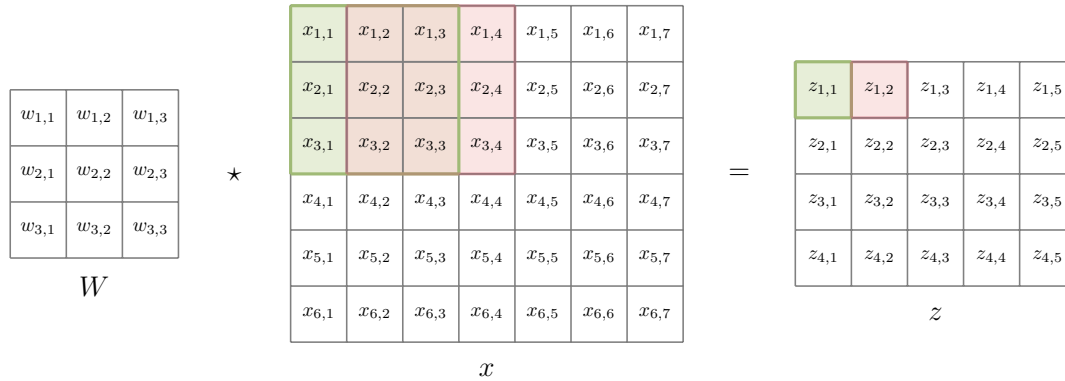
$$z_{i',j'} = (W \star x)_{i',j'} = \sum_{i=0}^{K_h-1} \sum_{j=0}^{K_v-1} w_{K_h-i, K_v-j} x_{i'+i, j'+j}, \quad (5.3)$$

for $i' = 1, \dots, M_h - K_h + 1$ and $j' = 1, \dots, M_v - K_v + 1$. The output has dimension

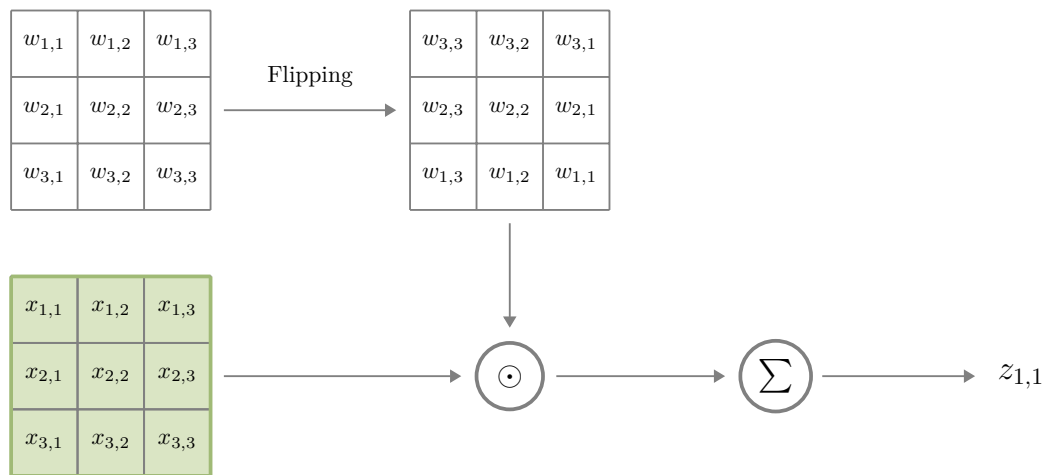
$$(M_h - K_h + 1) \times (M_v - K_v + 1). \quad (5.4)$$

Intuitively: the kernel W slides over all possible $K_h \times K_v$ windows of x , and at each position (i', j') computes the inner product of W with the corresponding patch. Figure 5.5 illustrates this operation.

¹Strictly this is the cross-correlation rather than convolution (no flipping of W); as discussed in Section 5.3, the two are equivalent for learning purposes.



(a) Schematic of 2D convolution between a 3×3 kernel W and a 6×7 image x , yielding a 4×5 output z .



(b) Computation of the single element $z_{1,1}$: element-wise product of the flipped kernel with the top-left 3×3 patch of x , then sum.

Figure 5.5: The 2D convolution operation (5.3).

Exercise 5.2.1 (Computing a 2D Convolution)

Let $W = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ be a 2×2 kernel and $x = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$ a 3×3 input. Use (5.3) to compute all four entries $z_{1,1}, z_{1,2}, z_{2,1}, z_{2,2}$ of $z = W \star x$, and verify that the output dimensions match (5.4).

5.3 From Fully Connected Layers to Convolutional Layers

We now derive the convolutional layer rigorously from a fully connected layer by imposing translation invariance and locality. This derivation shows precisely what is gained (parameter reduction) and lost (expressiveness) in the transition.

The General Fully Connected Layer on Image Data

Consider a single hidden layer of an FNN taking a greyscale image $x \in \mathbb{R}^{M_h^{[0]} \times M_v^{[0]}}$ as input and producing an output image $a^{[1]} \in \mathbb{R}^{M_h^{[1]} \times M_v^{[1]}}$. Indexing neurons by pixel coordinates (i', j') , the pre-activation is (from Week 2 notation):

$$z_{i',j'}^{[1]} = \sum_{(i,j)} w_{(i',j'),(i,j)}^{[1]} x_{i,j} + b_{i',j'}^{[1]}. \quad (5.5)$$

The weight tensor $w^{[1]}$ has $(M_h^{[1]} M_v^{[1]}) \times (M_h^{[0]} M_v^{[0]})$ entries — unconstrained.

Step 1: Imposing Translation Invariance

Translation invariance requires that shifting x by $(\Delta i, \Delta j)$ shifts $z^{[1]}$ by the same amount. Mathematically, the weight must depend only on the *offset* $(i'' - j'') = (i - i', j - j')$ rather than the absolute output location (i', j') . Re-indexing with $(i'', j'') = (i - i', j - j')$, equation (5.5) becomes

$$z_{i',j'}^{[1]} = \sum_{(i'',j'')} w_{i'',j''} x_{i'+i'',j'+j''} + b, \quad (5.6)$$

where the weights $w_{i'',j''}$ and bias b are now *shared across all output pixels*. This is **parameter sharing**. The weight tensor of dimension $(M_h^{[1]} M_v^{[1]}) \times (M_h^{[0]} M_v^{[0]})$ has been replaced by a much smaller set $\{w_{i'',j''}\}$ — one weight per offset.

Step 2: Imposing Locality

Locality requires that $z_{i',j'}^{[1]}$ depends only on a $K_h \times K_v$ neighbourhood of (i', j') in x . We enforce this by setting $w_{i'',j''} = 0$ for $i'' \notin \{0, \dots, K_h - 1\}$ or $j'' \notin \{0, \dots, K_v - 1\}$. Equation (5.6) then reduces to

$$z_{i',j'}^{[1]} = \sum_{i''=0}^{K_h-1} \sum_{j''=0}^{K_v-1} w_{i'',j''} x_{i'+i'',j'+j''} + b = (W \star x)_{i',j'} + b, \quad (5.7)$$

which is exactly the 2D convolution (5.3) (up to the kernel-flipping convention, which is irrelevant for learning since we can re-index W). The full convolutional layer is then

$$a^{[1]} = S^{[1]}(W \star x + b), \quad (5.8)$$

where $S^{[1]}$ is applied element-wise (typically ReLU). The kernel $W \in \mathbb{R}^{K_h \times K_v}$ and scalar bias b are the only learned parameters — a reduction from $O(M_h M_v \cdot M_h M_v)$ to $O(K_h K_v)$.

Example 5.3.1

For $M_h^{[0]} \times M_v^{[0]} = 224 \times 224$ input and $K_h \times K_v = 3 \times 3$ kernel, a fully connected layer with the same output size has $224^2 \times 224^2 + 224^2 \approx 2.5$ billion parameters. The convolutional layer has only $3 \times 3 + 1 = 10$ parameters. The convolutional layer is not less expressive in the *task-relevant* sense: for images, the same filter applied everywhere is exactly what we want.

Remark 5.3.1

Although the convolutional layer is written as a kernel sliding over patches, it remains a linear map: vectorising the input and output gives $\text{vec}(z) = T \text{vec}(x)$ for some matrix T . This T is *sparse* (most entries zero, from locality) and has *repeated entries* (from translation invariance: the same kernel weight reappears at every spatial position). The number of *distinct* non-zero entries in T equals the kernel size $K_h \times K_v$, while T itself has $(M_h^{[1]} M_v^{[1]}) \times (M_h^{[0]} M_v^{[0]})$ entries.

Exercise 5.3.1 (FC vs Conv Parameter Count)

Consider a single layer mapping a 4×4 greyscale image to a 3×3 greyscale output.

- As a fully connected layer (5.5) (one weight per output-input pixel pair, plus one bias per output neuron), how many learnable parameters does it have?
- As a convolutional layer with a 2×2 kernel and one shared bias, how many learnable parameters does it have? By what factor has the parameter count dropped?

5.4 Building a Convolutional Layer

Having derived the basic convolutional layer, we now introduce three important modifications — padding, stride, and dilation — and extend to multiple channels.

Padding

Each convolution in (5.4) reduces spatial dimensions by $K_h - 1$ and $K_v - 1$. After L_{conv} layers this compounds: a 3×3 kernel applied 16 times reduces each dimension by 32 pixels. **Padding** adds p_h rows and p_v columns of zeros around the input to control the output size. With padding (p_h, p_v) , the output dimension becomes

$$(M_h - K_h + p_h + 1) \times (M_v - K_v + p_v + 1). \quad (5.9)$$

Setting $(p_h, p_v) = (K_h - 1, K_v - 1)$ preserves the spatial dimensions. For odd-sized kernels it is common to set $p_h = \lfloor K_h/2 \rfloor$ symmetrically. Figure 5.6 illustrates padding with $(p_h, p_v) = (2, 2)$ for

a 3×3 kernel.

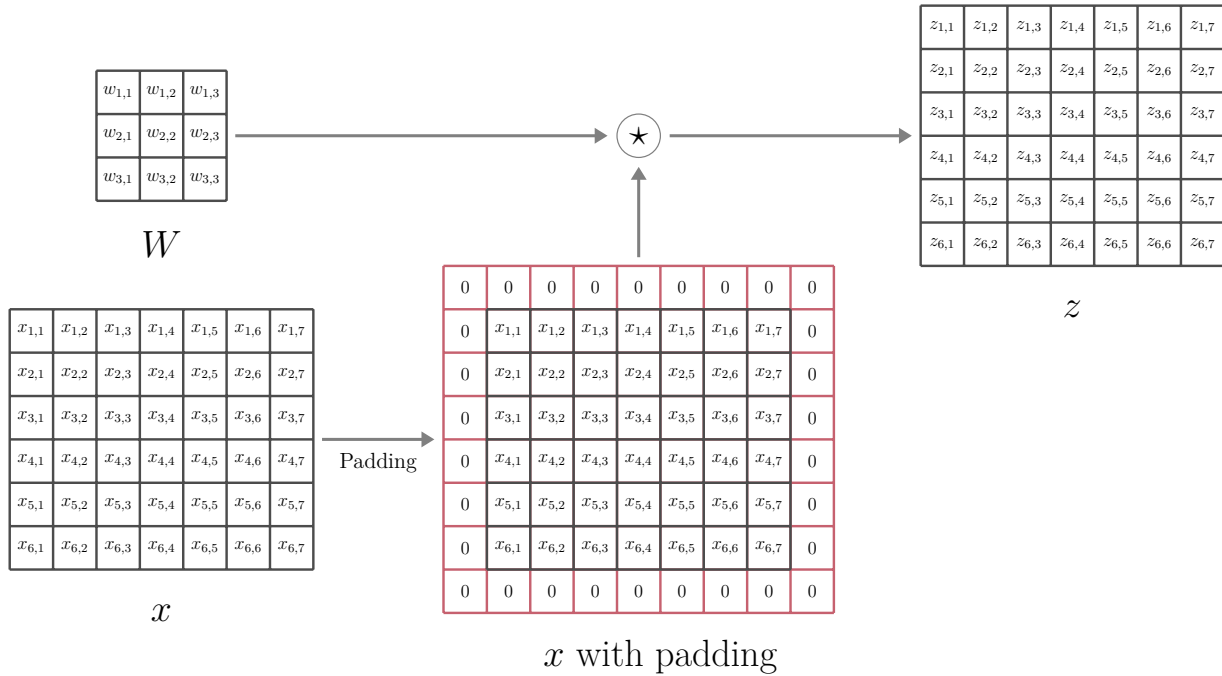


Figure 5.6: Padding with $(p_h, p_v) = (2, 2)$: zeros are added around the input so that the convolution output has the same spatial dimensions as the input.

Exercise 5.4.1 (Padding for Size Preservation)

For an input of size 28×28 and a 5×5 kernel (stride 1), find the padding (p_h, p_v) that makes the output also 28×28 . Verify using (5.9).

Stride

By default the kernel moves one pixel at a time (stride 1). A **stride** of $(s_h, s_v) > (1, 1)$ slides the kernel by s_h rows and s_v columns at each step, downsampling the output. With padding and stride, the output dimension is

$$\left(1 + \left\lfloor \frac{M_h - K_h + p_h}{s_h} \right\rfloor\right) \times \left(1 + \left\lfloor \frac{M_v - K_v + p_v}{s_v} \right\rfloor\right). \quad (5.10)$$

Stride offers a computationally efficient alternative to pooling for spatial downsampling. Figure 5.7 shows stride $(5, 4)$ for illustration (in practice, stride 2 is most common).

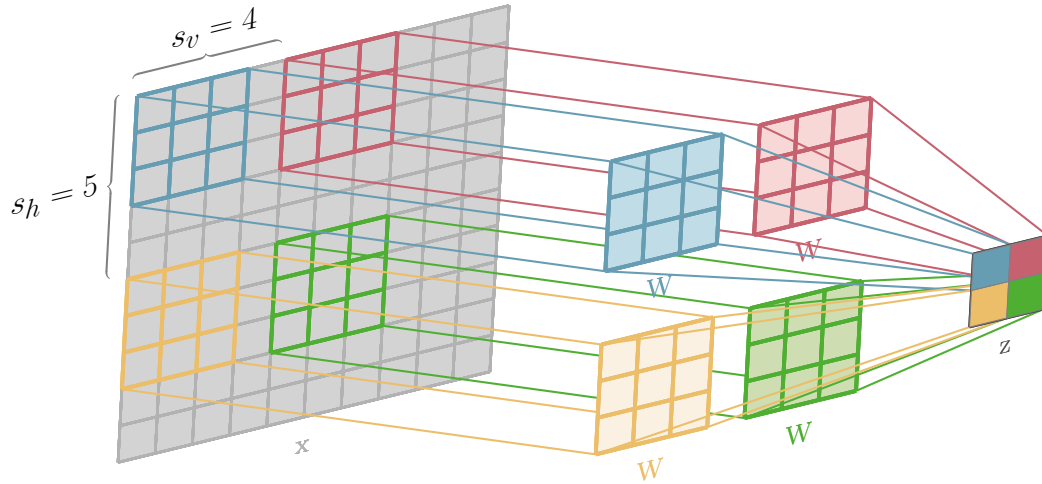


Figure 5.7: Stride $(s_h, s_v) = (5, 4)$: the kernel jumps 5 rows and 4 columns at each step.

Exercise 5.4.2 (Output Size with Stride)

For an input of size 32×32 , a 3×3 kernel, padding $(p_h, p_v) = (2, 2)$, and stride $(s_h, s_v) = (2, 2)$, compute the output spatial size using (5.10).

Dilation

Dilation increases the **receptive field** — the region of the input that influences one output pixel — without adding parameters. A dilation factor (d_h, d_v) inserts $d_h - 1$ zeros between consecutive kernel rows (and $d_v - 1$ between columns), expanding a $K_h \times K_v$ kernel to cover a $(d_h(K_h - 1) + 1) \times (d_v(K_v - 1) + 1)$ region. Substituting the dilated kernel size into (5.10) gives the general output dimension formula:

$$M_h^{[1]} \times M_v^{[1]} = \left(1 + \left\lfloor \frac{M_h^{[0]} - d_h(K_h - 1) - 1 + p_h}{s_h} \right\rfloor \right) \times \left(1 + \left\lfloor \frac{M_v^{[0]} - d_v(K_v - 1) - 1 + p_v}{s_v} \right\rfloor \right). \quad (5.11)$$

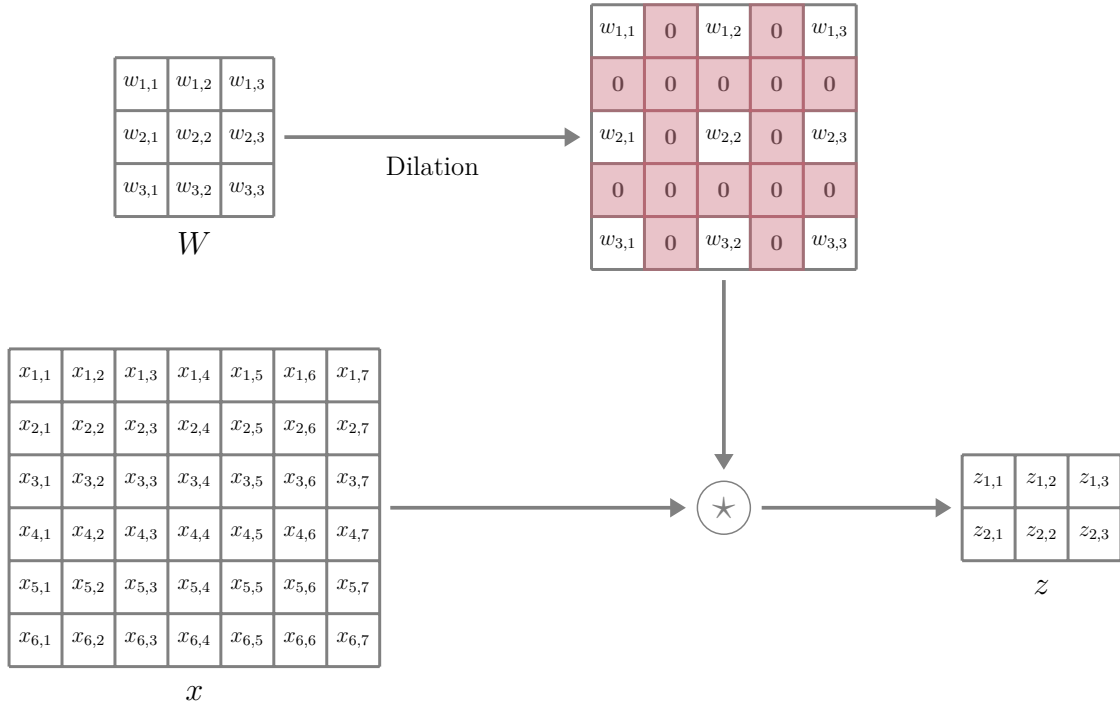


Figure 5.8: Dilation $(d_h, d_v) = (2, 2)$: a 3×3 kernel spans a 5×5 receptive field by inserting zeros between kernel elements.

Exercise 5.4.3 (Dilated vs Standard Kernel)

A 3×3 kernel with dilation $(d_h, d_v) = (2, 2)$ covers what spatial region of the input? Compare the parameter count of this dilated 3×3 kernel against a non-dilated kernel that covers the same region.

Multiple Input and Output Channels

A colour image has $M_c^{[0]} = 3$ channels (R, G, B). More generally, any input to a convolutional layer is a **tensor** of dimension $M_c^{[0]} \times M_h^{[0]} \times M_v^{[0]}$. To handle $M_c^{[0]}$ input channels, the kernel is also a tensor $W \in \mathbb{R}^{M_c^{[0]} \times K_h \times K_v}$. The convolution sums over all input channels:

$$z^{[1]} = \left(\sum_{i=1}^{M_c^{[0]}} W_{(i)} \star x_{(i)} \right) + b, \quad (5.12)$$

where $W_{(i)}$ and $x_{(i)}$ are the i th channel slices of W and x respectively (Figure 5.9).

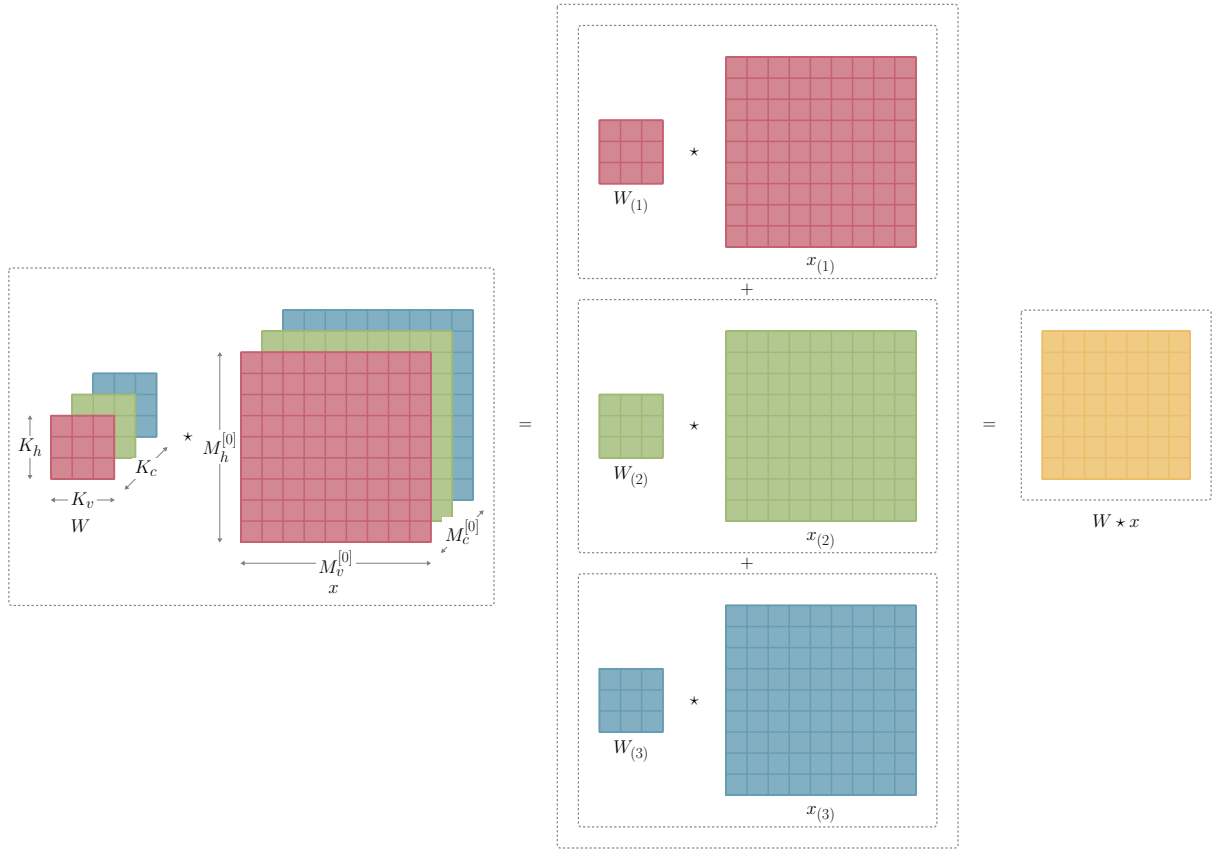


Figure 5.9: Volume convolution with $M_c^{[0]} = 3$ input channels. A $3 \times K_h \times K_v$ kernel convolves with each channel and the results are summed to produce a single output matrix.

To produce $M_c^{[1]}$ output channels (also called **feature maps**), we apply $M_c^{[1]}$ independent kernels $W_{(j)}$, each with its own bias $b_{(j)}$:

$$a_{(j)}^{[1]} = S^{[1]} \left(\left(\sum_{i=1}^{M_c^{[0]}} W_{(j),(i)} \star x_{(i)} \right) + b_{(j)} \right), \quad j = 1, \dots, M_c^{[1]}. \quad (5.13)$$

The output is a tensor of dimension $M_c^{[1]} \times M_h^{[1]} \times M_v^{[1]}$ (Figure 5.10). The total number of learned parameters per convolutional layer is

$$M_c^{[1]} \cdot (M_c^{[0]} \cdot K_h \cdot K_v + 1). \quad (5.14)$$

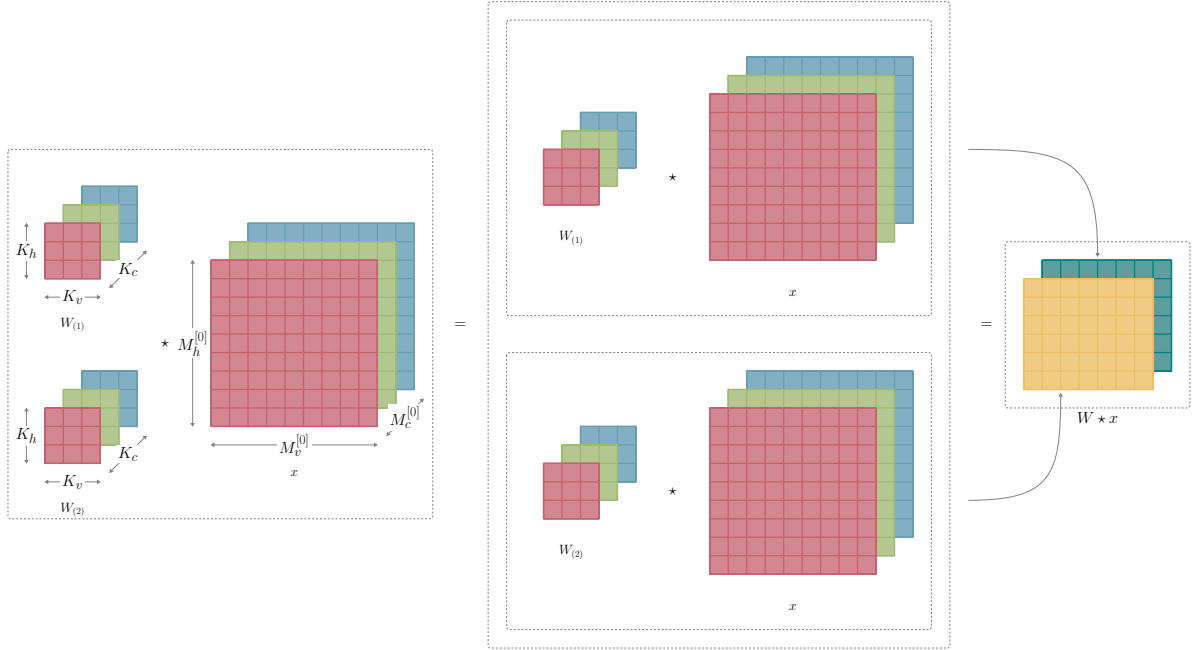


Figure 5.10: A convolutional layer with 3 input channels and 2 output channels. Each output channel is produced by a separate kernel applied to all input channels.

5.5 CNN Architectures: Pooling, VGG19, and ResNets

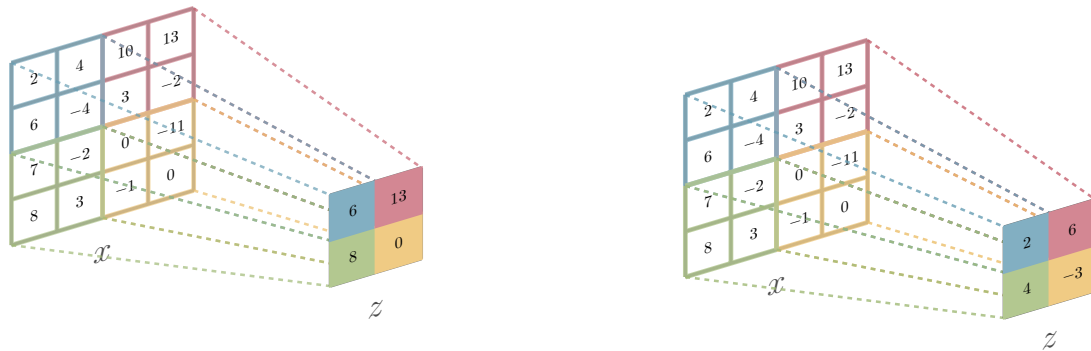
A full CNN stacks convolutional layers, non-trainable **pooling** layers, and finally dense layers. We examine each component and then study VGG19 in detail.

Pooling Layers

Pooling layers reduce the spatial dimensions of the activation tensor while preserving the number of channels. They have no learned parameters. Given a window $\mathcal{I}_{(i,k)}$ of pixel coordinates at output location (i,k) , the two standard pooling operations are

$$\left[a_{(j)}^{[\ell]} \right]_{i,k} = \begin{cases} \max_{(i',k') \in \mathcal{I}_{(i,k)}} \left[a_{(j)}^{[\ell-1]} \right]_{i',k'}, & (\text{max-pooling}) \\ \frac{1}{|\mathcal{I}_{(i,k)}|} \sum_{(i',k') \in \mathcal{I}_{(i,k)}} \left[a_{(j)}^{[\ell-1]} \right]_{i',k'}, & (\text{average-pooling}) \end{cases} \quad (5.15)$$

applied independently to each channel j . A typical 2×2 max-pool with stride 2 halves each spatial dimension (Figure 5.11).



(a) Max-pooling: retains the maximum in each 2×2 window.

(b) Average-pooling: averages each 2×2 window.

Figure 5.11: Pooling with a 2×2 non-overlapping window.

Receptive field growth. The **receptive field** of a neuron at layer ℓ is the set of input pixels that influence its value. With each convolutional layer of kernel size $K \times K$ and each pooling layer of size $P \times P$, the receptive field grows. Deeper neurons in the network depend on larger regions of the input, capturing progressively more global structure. Pooling accelerates this growth.

VGG19: A Complete Example

VGG19 has $L = 24$ total layers of which $L_{\text{train}} = 19$ are trainable: $L_{\text{conv}} = 16$ convolutional and $L_{\text{dense}} = 3$ dense; $L_{\text{pool}} = 5$ are max-pooling layers. All kernels are 3×3 with padding $(2, 2)$ and stride $(1, 1)$ — so convolutional layers preserve spatial dimensions. All pooling layers use 2×2 windows with stride $(2, 2)$, halving spatial dimensions at $224 \rightarrow 112 \rightarrow 56 \rightarrow 28 \rightarrow 14 \rightarrow 7$. The number of channels doubles after certain pooling layers, growing from $3 \rightarrow 64 \rightarrow 128 \rightarrow 256 \rightarrow 512$. A condensed view:

Layers	Type	Output dim.	Parameters
1–2	Conv $\times 2$	$64 \times 224 \times 224$	38,720
3	Max-pool	$64 \times 112 \times 112$	0
4–5	Conv $\times 2$	$128 \times 112 \times 112$	221,440
6	Max-pool	$128 \times 56 \times 56$	0
7–10	Conv $\times 4$	$256 \times 56 \times 56$	2,065,408
11	Max-pool	$256 \times 28 \times 28$	0
12–15	Conv $\times 4$	$512 \times 28 \times 28$	7,079,936
16	Max-pool	$512 \times 14 \times 14$	0
17–20	Conv $\times 4$	$512 \times 14 \times 14$	9,439,232
21	Max-pool	$512 \times 7 \times 7$	0
Flatten to 25,088-dim vector			
22–23	Dense $\times 2$	4,096	119,545,856
24	Dense (soft-max)	1,000	4,097,000
Total			143,487,592

Table 5.1: Condensed VGG19 architecture. The vast majority of parameters reside in the final dense layers. Using (5.14), each 3×3 convolutional layer has only $9M_c^{[\ell-1]}M_c^{[\ell]} + M_c^{[\ell]}$ parameters.

Inception Modules and 1×1 Convolutions

VGG19 fixes a single kernel size (3×3) at every layer. **Inception networks** (Szegedy et al., 2015) ask: at any one layer, why commit to one scale? An **inception module** processes its input through several parallel paths of *different* kernel sizes (1×1 , 3×3 , 5×5 , plus a max-pool) and concatenates the resulting feature maps along the channel dimension (Figure 5.12). The module thus learns features at multiple spatial scales *simultaneously*, leaving the optimiser to decide which scale matters at each layer.

A key building block of the inception module is the 1×1 **convolution**, which has spatial extent 1×1 but mixes information across channels. With $M_c^{[\ell-1]}$ input channels and $M_c^{[\ell]}$ output channels, a 1×1 convolution is a $M_c^{[\ell]} \times M_c^{[\ell-1]}$ matrix applied independently at every spatial position: it performs *channel-wise* dimension reduction (or expansion) without touching spatial structure. The Inception module uses 1×1 convolutions to reduce the channel dimension before the expensive 3×3 and 5×5 paths, keeping the parameter and compute budget low.

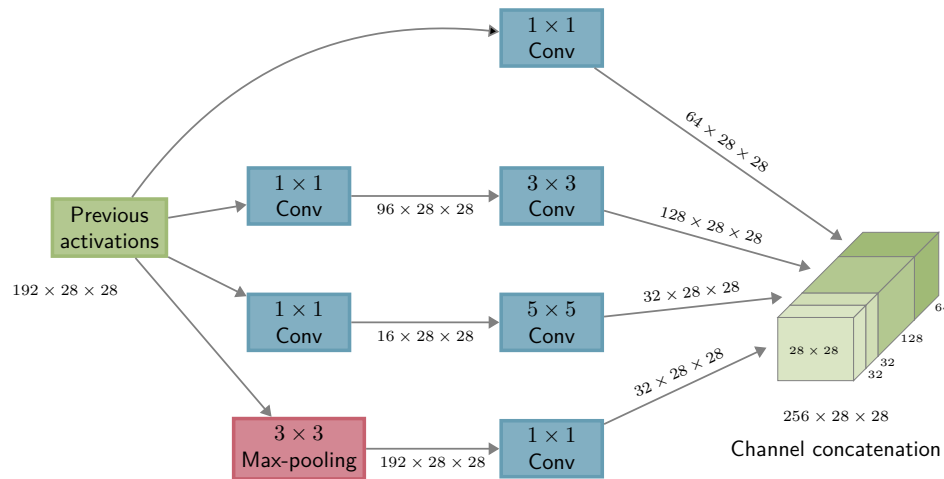


Figure 5.12: One form of an inception module: the input is processed in parallel by 1×1 , 3×3 , 5×5 convolutions and a 3×3 max-pool, and the outputs are concatenated along the channel dimension. The 1×1 convolutions before the larger kernels act as channel-wise dimension reduction, keeping the parameter count manageable.

Residual Connections and ResNets

In theory, adding more layers to a network should never increase training error (one can always set extra layers to the identity). In practice, very deep networks ($\gtrsim 20$ layers) suffer from *degradation*: training error rises as depth increases, partly due to the vanishing gradient problem discussed in Week 3.

ResNets (He et al., 2015) address this with **shortcut connections**: the input $a^{[\ell]}$ is added directly to the output of a block of layers to form $a^{[\ell+\tilde{\ell}]} = r(a^{[\ell]}) + a^{[\ell]}$, where $r(\cdot)$ is the block's transformation (Figure 5.13). The key insight is that learning $r(u) \approx 0$ is much easier than learning $f(u) \approx u$ directly: gradient information for $a^{[\ell]}$ bypasses the block, alleviating vanishing gradients. ResNets enabled training of networks with 50, 100, and even 1000+ layers.

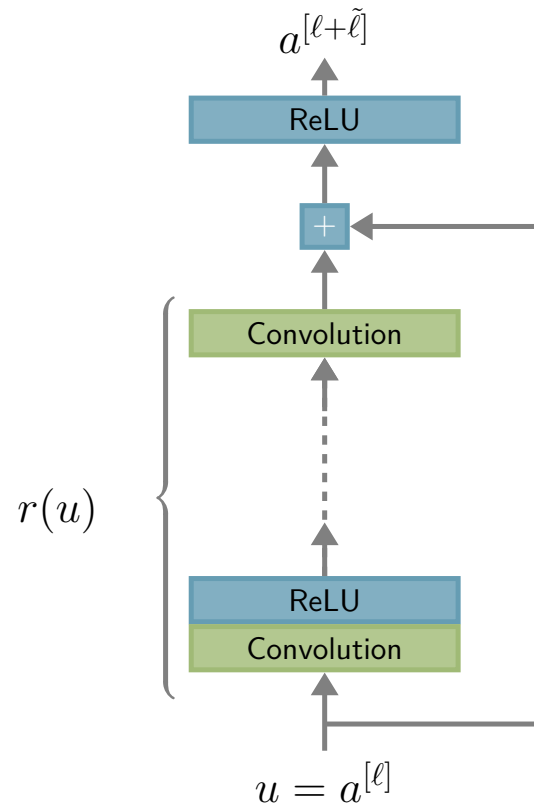


Figure 5.13: A residual (shortcut) connection: the block learns the residual $r(u)$ while the shortcut adds back the original input, making learning $r(u) \approx 0$ sufficient for the block to act as identity.

Architecture Landscape

Inception, ResNets, and their descendants (DenseNet, EfficientNet, ConvNeXt, ...) define a Pareto front in the accuracy / parameter / FLOPs space. Figure 5.14 shows how the EfficientNet family achieves the same top-1 ImageNet accuracy as earlier architectures while using a fraction of the parameters: the line of "compound scaling" of width, depth, and resolution traces out a clean efficiency frontier.

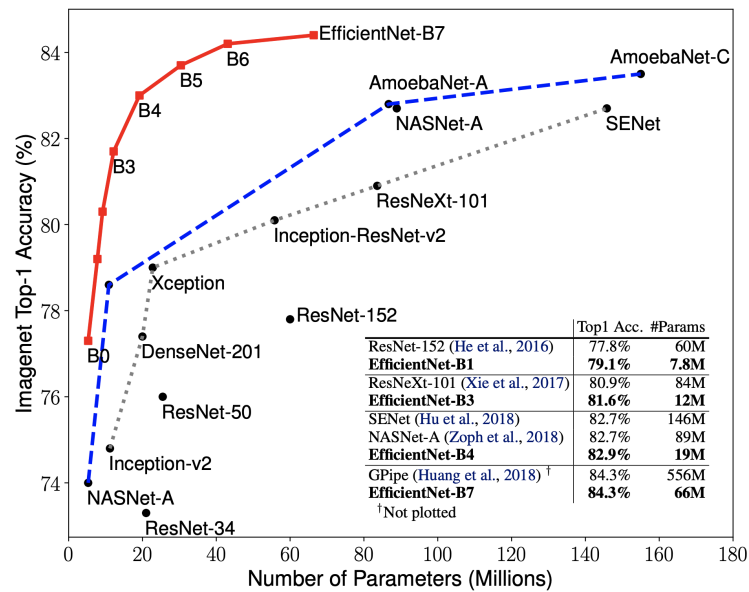


Figure 5.14: Top-1 ImageNet accuracy versus parameter count for several CNN architecture families. EfficientNet scales width, depth, and input resolution jointly with a single coefficient, achieving the same accuracy as much larger predecessors at a fraction of the parameter count.

Exercise 5.5.1 (VGG19 Analysis and the ResNet Gradient)

- Using (5.14), verify the parameter count for layers 1 and 2 of VGG19 in Table 5.1: layer 1 has $M_c^{[0]} = 3$, $M_c^{[1]} = 64$, 3×3 kernels; layer 2 has $M_c^{[0]} = 64$, $M_c^{[1]} = 64$. What fraction of VGG19's ≈ 143.5 million total parameters do these two layers account for?
- Dense layer 22 maps a vector of length 25,088 to 4,096 outputs. Using the dense-layer formula $N^{[\ell-1]} \cdot N^{[\ell]} + N^{[\ell]}$, compute its parameter count and determine what percentage of VGG19's total parameter budget it consumes.